CS228 Logic for Computer Science
Exercise Problem Set 8

Part 1

1. Recall the theory LRA (Linear Real Arithmetic): its universe is $\mathbb{R}$, its vocabulary is $V_T = \{0, 1, +, \leq, =\}$, and every symbol is interpreted as usual. Only *linear* combinations are allowed, i.e., there is no product of two variables. As in the lectures, $\geq$, $<$, $-$, numerals such as 10, and $c \cdot x$ for a numeral c (e.g., $2x$ for $x + x$) are abbreviations.

   (a) Which of the following are quantifier-free LRA formulas? For those that are not, explain why.

   (i) $x \geq 5 \wedge 2x - y = 3$

   (ii) $x \cdot y \geq 1$

   (iii) $(3x + 2y < 7) \vee \neg(x \leq y)$

   (iv) $\exists y\, (x \leq y)$

   (b) Translate each of the following statements about real numbers x and y into a quantifier-free LRA formula: φ_1 for S_1 and φ_2 for S_2.

   S_1: The sum of x and y is 10, x is at least 3 more than y, and either x is at most 6 or y is at most 3.

   S_2: x is at least twice y, y is at least 3, and either x is at most 5 or the sum of x and y is at most 8.

   Here x and y are arbitrary real numbers (not necessarily integers or positive), "either ... or" is the *inclusive* or (at least one of the two holds, possibly both), and "at most" and "at least" are non-strict (they allow equality).

   (c) Is the sentence $\exists x\, \exists y\, \varphi_1$ true in LRA? What about $\exists x\, \exists y\, \varphi_2$? Justify your answers by finding values of x, y or by showing that none exist. Why is $\exists x\, \exists y\, \varphi$ true in LRA exactly when φ is T-satisfiable?

2. Recall DPLL(T) from Lecture 18. Consider the formulas φ_1 and φ_2 that you wrote in Problem 1(b).

   (a) Write down the Boolean skeleton of φ_1 and of φ_2. (Number the theory atoms of each formula as $p_1, p_2, \ldots$ in the order in which they are mentioned in S_1, respectively S_2, and keep each atom as written, e.g., $x \geq y + 3$ is a single atom.)

   (b) Run DPLL(T) on φ_1 and on φ_2 to check your answers to Problem 1(c), using the Boolean skeletons from (a). For each run, list the decisions, propagations, calls to the T-solver and backtracking steps. Do not learn clauses: when the T-solver reports a conflict, simply backtrack and flip the most recent decision that has not been flipped yet. If there is no such decision, the answer is UNSAT. (Always propagate before deciding; when you must decide, pick the first undecided atom and set it to true. If an atom is set to false, the T-solver receives its negation, e.g., $x > 6$ for $\neg(x \leq 6)$.)

3. Read an n-bit vector $w = (b_{n-1}, \ldots, b_0)$ as the unsigned number $[w] = \sum_{i=0}^{n-1} b_i 2^i \in \{0, \ldots, 2^n - 1\}$. In the theory BV, addition is modulo 2^n: $[u + v] = ([u] + [v]) \bmod 2^n$, and $u <_u v$ means $[u] < [v]$.

(a) Prove that for every $n \geq 1$,

$$w + w <_u w \iff [w] \geq 2^{n-1} \iff b_{n-1} = 1.$$

(b) How many n-bit vectors satisfy $w + w <_u w$? List them for $n = 4$ and check the example $w = 1100$.

(c) Prove that $w + w = w$ holds if and only if $w = 0 \cdots 0$.

4. Recall the simple programming language from Lecture 19 (assignments, sequential composition, **if-else** and **while**). As in the lecture, all variables hold integers, and assertions are first-order formulas over the integers. Write a program for each of the following tasks.

(a) Compute $3^n + 2$ from n (input n, output y).

(b) Find the greatest y whose square is less than x (input x, output y).

(c) Compute the quotient q and the remainder r of dividing x by y (inputs x and y, outputs q and r).

(d) For each of your three programs P, write down a Hoare triple $\{\varphi\}\, P\, \{\psi\}$ that you would want to prove in order to show that P does its task. You may choose a precondition so that the triple becomes true. If your program overwrites one of its inputs, use a logical variable for the initial value of that input. (As with $x!$ in the lecture, you may use 3^n in assertions.)

5. Recall the following definitions from Lecture 19. A state α assigns an integer to every program variable. For a program P and assertions φ, ψ (first-order formulas over the integers):

- $\models_{par} \{\varphi\}\, P\, \{\psi\}$ (*partial correctness*) if for every state satisfying φ, *if* P terminates, the resulting state satisfies ψ;
- $\models_{tot} \{\varphi\}\, P\, \{\psi\}$ (*total correctness*) if for every state satisfying φ, P is guaranteed to terminate and the resulting state satisfies ψ.

For each triple below, decide whether it holds under partial correctness and under total correctness. Whenever one fails, give a starting state that witnesses the failure.

(a) $\{x > 0\}\ \ y := x * x\ \ \{y > x\}$

(b) $\{\top\}$ **if** $(x \leq y)$ $\{z := y\}$ **else** $\{z := x\}$ $\{z \geq x \wedge z \geq y\}$

(c) $\{\top\}$ **while** $(\top)$ $\{x := x + 1\}$ $\{x = 0\}$

(d) $\{x = 5\}$ **while** $(x \neq 0)$ $\{x := x - 2\}$ $\{\bot\}$

(e) $\{\exists z\, (z + z = x)\}\ \ x := x + 1\ \ \{\exists z\, (z + z = x)\}$

Part 2

6. Recall that a theory T fixes a vocabulary V_T and a single intended V_T-structure $\mathcal{M}_T$. A quantifier-free V_T-formula φ is *T-satisfiable* if there is an assignment α of values in the universe of $\mathcal{M}_T$ to the variables of φ such that $\mathcal{M}_T, \alpha \models \varphi$. The *$T$-satisfiability problem* (SMT) asks whether a given φ is T-satisfiable.

The *Boolean skeleton* φ^B of φ is obtained by replacing each distinct theory atom a_i of φ by a fresh propositional variable p_i.

(a) Prove that if φ is T-satisfiable, then φ^B is satisfiable.

(b) Show that the converse is false: give a T-unsatisfiable φ (for $T = \text{LRA}$) whose skeleton φ^B is satisfiable. What does this say about the role of the T-solver in DPLL(T)?

7. Let φ be a quantifier-free V_T-formula and let $L = \{\ell_1, \ldots, \ell_k\}$ be a set of theory literals (atoms or negated atoms) that is T-unsatisfiable, i.e. no assignment makes all of $\ell_1, \ldots, \ell_k$ true in $\mathcal{M}_T$. Let

$$C := \neg\ell_1 \vee \neg\ell_2 \vee \cdots \vee \neg\ell_k.$$

   (a) Prove that C is T-*valid*: every assignment satisfies C in $\mathcal{M}_T$.

   (b) Prove that φ and $\varphi \wedge C$ have exactly the same satisfying assignments in $\mathcal{M}_T$. Conclude that they are T-equisatisfiable.

   (c) Explain why this justifies DPLL(T) adding the skeleton of C as a learned clause. Run DPLL(T) on the LRA formula $(x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 1.5)$, with the first decision making $x \leq 0$ true. Identify L and C for the first T-conflict.

8. Use the definitions of $\models_{par}$ and $\models_{tot}$ from Problem 5. Programs in our language are deterministic: from a given state, P either runs forever or terminates in exactly one final state. Prove the following directly from the definitions.

   (a) If $\models_{tot} \{\varphi\}\, P\, \{\psi\}$ then $\models_{par} \{\varphi\}\, P\, \{\psi\}$. Give an example showing that the converse fails.

   (b) If $\models_{par} \{\varphi\}\, P\, \{\psi\}$, and $\varphi' \to \varphi$ and $\psi \to \psi'$ are valid over the integers, then $\models_{par} \{\varphi'\}\, P\, \{\psi'\}$.

   (c) $\models_{par} \{\varphi\}\, P\, \{\bot\}$ holds if and only if P does not terminate from any state satisfying φ.

   (d) $\models_{tot} \{\varphi\}\, P\, \{\top\}$ holds if and only if P terminates from every state satisfying φ.

9. For an integer expression E and a state α, let $[\![E]\!]_\alpha$ be the value of E in α, and let $\alpha[x \mapsto n]$ be the state that agrees with α except that it maps x to n. For an assertion ψ, $\psi[E/x]$ is ψ with every free occurrence of x replaced by E (renaming bound variables first, so that no variable of E gets captured). Running $x := E$ from α always terminates, in the state $\alpha[x \mapsto [\![E]\!]_\alpha]$.

   (a) Prove that for every term t: $[\![t[E/x]]\!]_\alpha = [\![t]\!]_{\alpha'}$, where $\alpha' := \alpha[x \mapsto [\![E]\!]_\alpha]$.

   (b) Prove the *substitution lemma*: $\alpha \models \psi[E/x]$ if and only if $\alpha' \models \psi$.

   (c) Conclude that $\models_{tot} \{\varphi\}\ x := E\ \{\psi\}$ holds if and only if the formula $\varphi \to \psi[E/x]$ is valid over the integers.

   (d) Use (c) to decide $\{x > 2\}\ y := x * x\ \{y > 4\}$ and $\{x > -3\}\ y := x * x\ \{y > 0\}$. Explain how an SMT solver could decide such a triple, and what a model returned by the solver means.

10. A familiar command missing from our language is the *for-statement*. It may be used to sum the elements in an array, for example, by programming as follows:

```
s = 0;
for (i = 0; i ≤ max; i = i + 1) {
   s = s + a[i];
}
```

    After performing the initial assignment $s = 0$, this executes $i = 0$ first, then executes the body $s = s + a[i]$ and the incrementation $i = i + 1$ continually until $i \leq max$ becomes false. Explain how **for** $(C_1;\ B;\ C_2)\ \{C_3\}$ can be defined as a derived program in our core language.